Laboratory Work 5

Working with Files, Folders, and Packages

Goal: To reinforce working with the file system (`pathlib`, `shutil`, `hashlib`), reading and writing files of various formats and encodings, processing data in `pandas`, and designing Python code as modules and a package.

To choose your variant number, run the expression:

```
N = ord("XXX") % 3 + 1,
```

where XXX is the first capital letter of your first name in Latin transliteration. For example, for James Bond the variant number is N = 3

Common Package Requirements (for All Variants)

1. The package must have `__init__.py`, at least 4 modules, and one subpackage with its own `__init__.py`.
2. Use both absolute and relative imports (`from .utils import ...`).
3. In every module, define `__all__`; in `__init__.py` export the package's public API and `__version__`.
4. Entry point: `__main__.py`, run with `python -m <package>` ..., arguments via `argparse`.
5. Build all paths using `pathlib.Path`; no hard-coded absolute paths.
6. Handle errors (file not found, corrupted line, unknown encoding) with exceptions; instead of `print` for service messages, use `logging`.
7. Save results to the `output/` folder (create it automatically): at least one CSV and one `JSON` report.
8. At least 5 pytest tests on the data in `tests_data/tiny_*`; verify the full data against `expected_full.json`.
9. Add `requirements.txt` and `README.md` (package structure, how to run, example output).

Variant 1. Server Log Analyzer

Scenario

The server writes logs in two formats into date-based folders. You need to collect all logs into a single DataFrame, separate the malformed lines, and build reports on errors and IP address activity.

Input data (`variant1_logs/data/server_logs/`**)**

```
server_logs/
├── README.txt                  # not a log file, ignore
├── 2026-10-01/ web.log, api.log
├── 2026-10-02/ web.log, api.log
├── 2026-10-03/ web.log (windows-1251, Ukrainian messages),
api.log
├── 2026-10-04/ web.log (empty file), api.log
└── 2026-10-05/ web.log, api.log, web.log.bak   # ignore
.bak
```

File structure

- `web.log`: `2026-10-01 12:03:45 [ERROR] 192.168.1.10 Database timeout after 30s`
- `api.log` (JSON Lines): `{"ts": "2026-10-01T12:03:45", "level": "WARN", "ip": "10.0.0.5", "msg": "..."}`

Intentional problems: different formats; one file in windows-1251; an empty file; blank lines; garbage lines (Traceback..., truncated JSON); the WARN level in api.log must be normalized to WARNING; extra files (README.txt, .bak).

Recommended package structure

```
log_analyzer/
├── __init__.py
├── __main__.py           # CLI: scan | report
├── config.py             # file patterns, levels, candidate encodings
├── scanner.py            # search for *.log (rglob), filtering out extras
├── readers.py            # reading with encoding detection (utf-8 -> win-1251)
├── parsers/              # subpackage
│   ├── __init__.py
│   ├── web.py            # regex parser for the text format
│   └── api.py            # JSON parser
├── analytics.py          # aggregations over the DataFrame
└── reports.py            # export CSV/JSON/Excel
```

Tasks

1. **Scanning:** find all *.log files recursively and return a list of paths (excluding `.bak` and `.txt`).
2. **Reading:** read each file with automatic encoding detection; an empty file must not crash the program.

3. **Parsing:** each format in its own module of the parsers subpackage; do not lose malformed lines, but count and log them (`WARNING` with the file and line number).
4. **DataFrame:** columns `timestamp` (type `datetime`), `level`, `ip`, `message`, `service`, `source_file`.
5. **Analytics:** number of records by level; number of `ERROR`+`CRITICAL` per day; top 5 IPs; the hour with the most errors.
6. **Reports:** `output/summary.json, output/errors_by_day.csv, output/top_ips.csv, output/malformed_lines.csv`.
7. **CLI:** `python -m log_analyzer report --input data/server_logs --output output --level ERROR`.

Variant 2. Personal Finance Manager

Scenario

Expenses are stored in exports from two banks and in manual notes about cash. The formats differ and the data partly overlaps. You need to merge everything into a single DataFrame, categorize it using rules from JSON, and build monthly reports.

Input data (`variant2_finance/data/`)

```
data/
├── bank_exports/
│   ├── monobank_2026-08.csv     # cp1251, ';' delimiter, decimal comma, date
DD.MM.YYYY
│   ├── monobank_2026-09.csv     # same + 2 overlapping rows with August
│   ├── privatbank_2026-09.csv   # UTF-8, ',' delimiter, ISO date, separate currency
column (UAH/USD)
│   └── cash_notes.json          # cash: expenses as POSITIVE numbers, category
already set
└── config/
    ├── categories_rules.json    # keywords -> category
    └── exchange_rates.json      # USD -> UAH = 41.5
```

Intentional problems: different encodings, delimiters, and date and number formats; exact duplicates within files; overlap between files by month; rows with an empty or non-numeric amount; a non-existent date (`31.02.2026`); inconsistent case and extra spaces in descriptions (`SILPO #12`, `uber`); expenses in USD; different amount signs in the bank and in the notes.

Recommended package structure

```
budget_tracker/
├── __init__.py
├── __main__.py            # CLI: summary | export
├── config.py              # paths, loading JSON configs
├── loaders/               # subpackage
│   ├── __init__.py        # registry of loaders by file
name
│   ├── monobank.py
│   ├── privatbank.py
│   └── cash.py
├── cleaning.py            # conversion to a unified schema,
duplicates, invalid rows
├── categorizer.py         # applying the rules
├── analytics.py           # pivot tables
└── reports.py             # write to
output/reports/YYYY/MM/
```

Tasks

1. Each loader returns a `DataFrame` with **a unified schema**: `date`, `description`, `amount_uah`, `currency`, `source`.
2. Combine using `pd.concat`; remove duplicates by the key "source + date and time + description (`strip`().`lower`()) + amount".
3. Do not silently discard invalid rows: save them to `output/rejected_rows.csv` with the reason.
4. Categorization: case-insensitive substring search; if there is no match — the "Інше" (Other) category; for cash, use the category already provided.
5. Convert USD to UAH using the rate from the config.
6. Reports: expenses by category, by month, a "month × category" pivot table (`pivot_table`), income separately. Save to the folders `output/reports/2026/08/` and `2026/09/` (create automatically).
7. CLI: `python -m budget_tracker summary --input` data `--month 2026-09`.

Variant 3. File Organizer with Duplicate Detection

Scenario

The "Downloads" folder is cluttered: various file types, copies with different names, empty files, system junk. You need to build an inventory in a DataFrame, find duplicates by content, build a report, and (in safe mode) sort the files into categories.

Input data (`variant3_files/data/`)

```
data/
├── config/extensions_map.json     # category -> extensions
└── messy_downloads/               # 44 files in nested folders (up to 4 levels)
    ├── Documents/…  Images/…  Media/…  Projects/…  Archives/…
    ├── Інсталятори/  old stuff/  tmp/  Data/
    ├── to_delete_empty/           # empty folder
    └── .DS_Store, Images/Thumbs.db, Documents/desktop.ini   # system, ignore
```

Intentional problems: Cyrillic characters and spaces in names; extensions in different case (`.PNG, .MP3, .PDF`); files without an extension (`README`, `Makefile`) and unknown extensions (`.xyz`, `.tmp`); 5 groups of duplicates with different names and folders (one group of three copies); **trap:** `photo (1).jpg` has the same size as `IMG_2041.jpg`, but different content, so comparing by size alone gives a false result; 3 empty files; an empty folder; system files.

Recommended package structure

```
file_organizer/
├── __init__.py
├── __main__.py            # CLI: scan | duplicates | organize
├── config.py              # loading extensions_map.json, list of system files
├── scanner.py             # recursive traversal, metadata -> DataFrame
├── hashing.py             # sha256 in chunks, without reading the whole file
├── duplicates.py          # grouping by (size -> hash)
├── mover.py               # moving via shutil, dry-run, name collisions
└── reports/               # subpackage
    ├── __init__.py
    ├── summary.py         # statistics by category
    └── export.py          # CSV/JSON
```

Tasks

1. `DataFrame` with columns `path`, `name`, `extension` (lowercase), `category`, `size_bytes`, `modified` (`datetime`). Skip system files.
2. **Report by category**: count and total size (`groupby`), the 5 largest files, the oldest and the newest file.
3. **Duplicates**: first group by size, then compute the hash only for candidates; empty files are not included in duplicate groups. Show the "lost" volume (size × (`copies – 1`)).
4. **Organization**: mode `--dry-run` (default) only prints the plan; with the flag `--apply`, files are moved to `organized`/<`Category`>/. On name collisions, add the suffix `_1, _2`. Nothing is deleted. Keep a move log in `output/moves_log.csv`.

5. **Safety**: the program works only in the specified folder and does not go beyond its boundaries; before `--apply` the student works with a copy of `messy_downloads`.
6. **CLI**: `python -m file_organizer organize --source data/messy_downloads --target organized --dry-run`.